

cert_analyzer_selfvalidation_test.sh

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z

Standing §11.4.135 regression guard for the Let's Encrypt Phase 3 cert-chain / expiry analyzer. It proves tests/letsencrypt/cert_analyzer.sh is a **self-validated analyzer** (§11.4.107(10)): it ACCEPTS every golden-GOOD certificate property AND REJECTS every golden-BAD one (expired / wrong-CA / wrong-host / substring-not-a-match). An analyzer that accepts its golden-bad fixture is a bluff gate — it would report "cert OK" for a broken or wrong certificate, the §11.4 PASS-bluff at the certificate-validation layer. The guard is client-AND-challenge-AGNOSTIC: it validates a resulting cert regardless of how it was obtained (Caddy / lego / certbot / Pebble / LE staging / LE prod), so it does not depend on the pending operator decisions (design §9: DNS-01 vs HTTP-01, Caddy vs lego).

Overview

Field	Value
Path	tests/regression/ cert_analyzer_selfvalidation_test.sh
Under test	tests/letsencrypt/cert_analyzer.sh (cert_not_expired / cert_days_remaining / cert_san_matches / cert_chain_roots_in / cert_renewal_due)
Fixtures	tests/letsencrypt/fixtures/*.pem (generated by gen_fixtures.sh)
Kind	Hermetic self-validation guard — no network, no ACME, no container boot
Anti-bluff anchors	§11.4.107(10), §11.4.50, §11.4.115, §11.4.135, §1.1
Exit codes	0 = PASS, 1 = FAIL

Prerequisites

- POSIX sh, plus openssl, GNU date -d, awk/sed/tr, mktemp on PATH.
- The golden fixtures under tests/letsencrypt/fixtures/ — the guard self-bootstraps them via gen_fixtures.sh when the tracked *.pem set is absent (§11.4.77 regeneration mechanism).
- No live service, no ACME endpoint, no credentials are required — the guard is entirely offline and deterministic via a pinned "now" epoch (2026-07-01T12:00:00Z), matching the FIXED validity windows baked into the fixtures (§11.4.50).

Usage examples

```
# GREEN standing guard (default): asserts the real analyzer
    accepts golden-good
# and rejects every golden-bad.
tests/regression/cert_analyzer_selfvalidation_test.sh

# RED reproduction (§11.4.115): asserts the pre-fix naive
    analyzers (presence-
# only, ignoring expiry; SAN substring match) WRONGLY accept a
    golden-bad — the
# bluffs the real analyzer closes.
RED_MODE=1 tests/regression/cert_analyzer_selfvalidation_test.sh
```

Run under the mandated host caps:

```
GOMAXPROCS=2 nice -n 19 ionice -c 3 tests/regression/
    cert_analyzer_selfvalidation_test.sh
```

What it asserts

The guard sources the REAL analyzer and the golden fixtures, pins now = 2026-07-01T12:00:00Z, and asserts the full accept/reject matrix:

Fixture / call	Property	Expected
cert_not_expired good_leaf @now	inside validity window	ACCEPT
cert_san_matches good_leaf proxy.test	exact SAN token	ACCEPT
cert_chain_roots_in good_leaf test_ca	issued by expected CA	ACCEPT
cert_renewal_due nearexpiry_leaf 30 @now	~4d left ≤ 30d threshold	ACCEPT (due)
cert_not_expired expired_leaf @now	NotAfter in the past	REJECT

Fixture / call	Property	Expected
cert_chain_roots_in wrongca_leaf test_ca	signed by a different CA	REJECT
cert_san_matches otherhost_leaf proxy.test	SAN is other.test	REJECT
cert_san_matches good_leaf oxy.test	substring, not a token	REJECT
cert_renewal_due good_leaf 30 @now	~59d left > 30d threshold	REJECT (not due)

The decisive anti-bluff assertions are the five REJECT rows: any change that lets a golden-bad property be ACCEPTED flips the guard to FAIL with a real assertion mismatch (MISMATCH(reject): ... analyzer ACCEPTED a golden-bad (BLUFF)) — not a parse error.

RED_MODE=1 runs faithful pre-fix replicas — a "presence-only" checker that skips the expiry gate, and a SAN **substring** matcher — against the golden-bad inputs and asserts they ACCEPT them (the expired leaf; oxy.test vs proxy.test), reproducing the two classic bluffs the real analyzer closes.

Edge cases

- **Absent / unparseable PEM** → the analyzer returns 2; the guard treats non-zero as REJECT (no false-PASS on a missing cert).
- **Issuance vs expiry orthogonality** — cert_chain_roots_in uses openssl verify -no_check_time, so an expired-but-correctly-issued cert still roots in its CA (chain ACCEPT) while cert_not_expired REJECTs it. The two concerns never mask each other.
- **Wildcard SAN** — *.proxy.test covers exactly one extra left label (a.proxy.test) but not the apex (proxy.test) nor two labels (a.b.proxy.test); exercised by the unit selftest, not this guard.
- **RED cannot reproduce** → the guard reports a §11.4.7 finding rather than a false PASS.

Internal behaviour

1. Resolve REPO_ROOT from the script location; create the gitignored evidence dir qa-results/regression/cert_analyzer_selfvalidation/.
2. Bootstrap the golden fixtures if absent (gen_fixtures.sh).
3. Source the REAL cert_analyzer.sh (never re-implement the functions).

4. Pin now to a fixed epoch and, per RED_MODE, either drive the real analyzer across the accept/reject matrix (GREEN) or drive the naive pre-fix replicas against the golden-bad inputs (RED).
5. Write a structured evidence file; print a single [PASS|FAIL] verdict line + the evidence path; exit 0/1.

§1.1 paired-mutation proof

Mutate `cert_not_expired()` in `cert_analyzer.sh` so it always returns 0 (accepts an expired cert). The GREEN guard then FAILs on `MISMATCH(reject): not_expired expired -> analyzer ACCEPTED a golden-bad (BLUFF)` (a real assertion mismatch, exit 1). Restore the library byte-identical (md5 match) and the GREEN guard PASSes again. This proves the guard is not a bluff gate.

Related scripts

- `tests/letsencrypt/cert_analyzer.sh` — the analyzer this guards.
- `tests/letsencrypt/cert_analyzer_selftest.sh` — the TAP unit self-validation matrix (37 cases) driving every function against golden-good + golden-bad.
- `tests/letsencrypt/fixtures/gen_fixtures.sh` — the hermetic openssl fixture generator (§11.4.77).
- `tests/lib/evidence.sh` — canonical `ab_pass_with_evidence / ab_skip_with_reason` (the house anti-bluff style this follows).
- `docs/design/LETSENCRYPT_HTTPS.md` §6 + `LETSENCRYPT_HTTPS_PLAN.md` Phase 3 — the design + plan that specify this analyzer.

Last verified

2026-07-01 — GREEN PASS (37/37 unit selftest + guard),
RED_MODE=1 PASS (reproduction), and the §1.1
mutation→FAIL→restore(md5-match
a06e0f89ee9993fe5de368be852e4eff) cycle captured hermetically (no
network, no ACME, no container boot).